

Revealing MonoT5’s Learning Mechanisms via Prompt-Token Adaptation

Marco Braga^{1,2}[0009–0004–7619–8399], Sean MacAvaney³[0000–0002–8914–2659],
Craig Macdonald³[0000–0003–3143–279X], and Gabriella
Pasi¹[0000–0002–6080–8170]

¹ University of Milano-Bicocca, Milan, Italy
`m.braga@campus.unimib.it`, `gabriella.pasi@unimib.it`

² Politecnico di Torino, Turin, Italy

³ University of Glasgow, Glasgow, Scotland, UK
`{sean.macavaney,craig.macdonald}@glasgow.ac.uk`

Abstract. Transformer-based cross-encoders, such as MonoT5, achieve state-of-the-art performance in several retrieval tasks. In particular, MonoT5 is based on a sequence-to-sequence architecture and trained on MS MARCO for passage re-ranking, where it predicts the relevance of a passage given an input query. In this paper, to understand what MonoT5 learned, we analyse the parameter updates during its training process. We observe that the largest shifts occur in a small set of parameters, i.e. less than 1% of the model, while the rest of the model remains relatively unchanged. Motivated by this finding, we propose Light-MonoT5, a parameter-efficient variant of MonoT5 that updates only this small set of parameters during training, and leaves the rest of the network unchanged. Extensive evaluation on both in- and out-domain benchmarks shows that Light-MonoT5 achieves statistically equivalent effectiveness compared to MonoT5. Since relevance can be captured by updating only a subset of T5 parameters, we hypothesise that MonoT5, which updates all the original model’s parameters, primarily learns to evaluate passage quality rather than explicitly assessing the relevance of a passage to the query. To test our hypothesis, we employ QT5, a T5-based quality estimation model, to prune low-quality passages before indexing. On the pruned collection, Light-MonoT5 achieves performance on par with MonoT5, indicating that MonoT5’s strong performance is largely attributable to quality assessment, with minimal adaptation required once low-quality content is removed.

Keywords: MonoT5 · Parameter Efficient Fine-Tuning · Neural IR

1 Introduction

The introduction of the transformer architecture [29] and the rise of Pre-trained Language Models (PLMs), such as T5 [25], have transformed the Information Retrieval (IR) field. Among them, MonoT5 [22], which fine-tunes T5 on the MS MARCO [21] dataset for document reranking, achieved state-of-the-art performance on various retrieval benchmarks. MonoT5 is a cross-encoder model [15],

which jointly encodes both query and document as a single input. Unlike traditional BERT-based retrieval models [23], MonoT5 is trained using a natural language prompt that explicitly encodes task-specific tokens, such as **Query**, **Document** and **Relevant**, to guide the model during fine-tuning. The model is then trained to generate **true** if the document is relevant to the query, and **false** otherwise. Despite its strong empirical performance, the inner mechanism of MonoT5 remains understudied. In particular, it is unclear how relevance signals are encoded during fine-tuning and which components of the model contribute most significantly to its behaviour. For these reasons, we analyse the parameter shifts in T5 when fine-tuned as MonoT5 on MS MARCO [21]. Our study reveals that the highest changes are not distributed equally across the model but in a relatively small number of parameters concentrated in the embeddings layer. In particular, the token embeddings with the highest shift are **Query**, **Document**, **Relevant**, **true**, **false**, which are the ones used in the prompt and to learn relevance. This analysis suggests that much of the retrieval-specific behaviour and relevance signal is directly encoded into these specific embeddings, while the majority of the model remains relatively unchanged.

Building on these insights, we propose a novel training strategy that updates only the embedding vectors associated with prompt tokens, leaving the rest of the model frozen. Our approach falls into the category of Parameter-Efficient Fine-Tuning (PEFT) techniques, which update only a small set of model parameters while fine-tuning a model on a downstream task. Indeed, our approach requires updating only 0.002% of the model’s parameters while preserving retrieval effectiveness. By focusing adaptation on prompt embeddings, our method enables efficient adaptation without the overhead of full-model fine-tuning. We refer to the model obtained by applying our training strategy to T5 as Light-MonoT5, highlighting its lightweight adaptation. We evaluate Light-MonoT5 on both in-domain (TREC DL 19 [10], DL 20 [9] and DL Hard [19]) and out-domain (BEIR [28]) benchmarks, showing that our approach achieves performance that is not statistically significantly different from MonoT5.

Based on our findings, we hypothesise that MonoT5 primarily learns to assess passage quality, as defined by Chen et al. [7], since the base T5 model [25] already learned semantic matching as one of its pertaining tasks. This hypothesis is motivated by the observation that updating only a small subset of parameters allows Light-MonoT5 to achieve performance comparable to full MonoT5 fine-tuning. To test our hypothesis, we leverage Quality T5 (QT5) [7], a T5-based model trained to predict passages that are unlikely to be relevant to any query, such as those containing repeated sentences, and prune them from the corpus. Light-MonoT5 applied to the pruned corpus achieves retrieval effectiveness on par with MonoT5. These results support our hypothesis: MonoT5’s strong performance can be largely attributed to its ability to recognise document quality, because once low-quality passages are removed, only minimal adaptation is sufficient to recover full retrieval effectiveness. To summarise, the contributions of this work are as follows:

- We analyse the behaviour of fine-tuning T5 on MS MARCO, showing that its parameter shifts are concentrated in the embeddings of prompt tokens.
- We propose a new parameter-efficient tuning strategy that adapts only prompt embeddings, reducing trainable parameters to 0.002% while preserving strong retrieval performance.
- We evaluate the effectiveness of our approach across diverse tasks and domains, showing its potential as a lightweight technique for training T5.
- Based on the previous contributions, we show how MonoT5 learns to classify low-quality documents instead of query-document relevance.

To help the reproduction of the results, we make our code publicly available⁴. The rest of the paper is organised as follows: Section 2 provides an overview of T5 and Parameter Efficient Fine-Tuning techniques, Section 3 shows the analysis on the T5 parameters while training it on MS MARCO, Section 4 describes in detail our proposed approach, i.e. Light-MonoT5. Next, Section 5 outlines the experimental setup used to evaluate our approach, Section 6 analyses the results of our evaluation and Section 7 contains concluding remarks.

2 Related Work

This section introduces previous work on T5 and its applications in the Information Retrieval (IR) field. Then, we provide a general overview of Parameter Efficient Fine-Tuning (PEFT) techniques.

T5. The architecture of T5 [25] is based on the transformer model [29], which consists of an encoder and a decoder. The encoder processes the input text, while the decoder generates the output text. Both components include an embedding layer followed by multiple stacked blocks (e.g., 12 in the T5-base configuration). Each block contains a self-attention mechanism, a two-layer feed-forward network, normalisation layers, and residual connections. In addition, the decoder incorporates a cross-attention mechanism that conditions the generation process on the encoder’s representations. T5 has been pre-trained as language model on the C4 dataset [25], which is a large-scale dataset of cleaned pages from the Web written in English. T5 is then further fine-tuned on a diverse set of tasks [25], including translation, question answering, and classification, with a maximum likelihood objective. To specify which task the model should perform, Raffel et al. [25] add a task-specific text prefix to the original input sequence before feeding it to the model. After processing the input text, the model generates new text or a single word corresponding to the target label. Based on T5, Nogueira et al. [22] proposed MonoT5, which is defined by fine-tuning T5 on MS MARCO [21] for document reranking. Given a query q and a document d , MonoT5 is trained to correctly identify if the document is relevant with respect to the query. Following the T5 fine-tuning strategy [25], Nogueira et al. [22] defined a new prompt to specify the task-specific instruction: **Query: [q] Document: [d] Relevant:**, where [q] and [d] are replaced with the query and document

⁴ anonymous.4open.science/Light-MonoT5

texts, respectively. The model has to answer **true** or **false**, which are the target labels. Recent studies [22,24] have shown that MonoT5 heavily relies on the task-specific prompt tokens it is trained with: while it can flexibly adapt to arbitrary label semantics [22], it is also highly vulnerable to adversarial prompt manipulations such as preemption, stuffing, and rewriting [24].

While MonoT5 is trained for learning if a document is relevant to a query, Cheng et al. [7] proposed Quality T5 (QT5), which is still a T5-based model, but has been fine-tuned to predict the quality of a passage, i.e. if it is unlikely to be relevant to any query. For example, passages with a large number of repeated tokens are more likely to be irrelevant to any query [7]. Different from MonoT5, QT5 is trained over MS MARCO triples but ignoring the query with the following prompt: **Document: [x] Relevant: [true/false]**. QT5 can be used to prune low-quality passages before indexing, reducing both computational memory and indexing time, while maintaining statistically equivalent effectiveness.

PEFT techniques. Despite its language abilities, fine-tuning T5 on a downstream task requires the training of millions of parameters (from 200M of T5-base to 740M of T5-large), which can be computationally expensive and time-consuming. To solve these issues, PEFT techniques have been proposed [3]. PEFT techniques are specifically designed to train only a subset of the model’s original parameters while adapting the model to a downstream task, and maintaining performance levels comparable to those achieved with traditional fine-tuning. One of the most popular approaches is LoRA [14], which updates the attention weights $W \in \mathbb{R}^{d \times k}$ as $W + \Delta W = W + BA$, where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ are the only parameters to be trained and $r \ll \min(d, k)$ is the low rank of matrix BA . While LoRA requires the definition of a new small set of parameters to be trained on (i.e. A, B), in our work, we focus on training a small subset of original model parameters, which allows us to be more efficient.

3 Preliminary Analysis of MonoT5

Recent works [4,16] define the parameter difference between a PLM and its fine-tuned version on a specific task as the task vector, which represents the knowledge learned by the model during the training and encodes all the information needed to perform well on the specific task. Task vectors have been applied in the field of IR to define domain-specific reranker models [4]. Inspired by these works, we analyse the task vector between T5-base and MonoT5-base to address the following research question:

- RQ0: Which parameters change more while fine-tuning MonoT5?

For each weight matrix $\theta_{MonoT5} \in \mathbb{R}^{n \times m}$ in each layer of MonoT5, we compute the task vector with the corresponding weight $\theta_{T5} \in \mathbb{R}^{n \times m}$ in T5 as $\Delta\theta = \theta_{MonoT5} - \theta_{T5}$. Let be $(\Delta\theta)_{i,j}$ the element in row i and column j of $\Delta\theta$, we calculate the magnitude of $\Delta\theta$ by applying the Frobenius norm [13]:

$$\|\Delta\theta\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^m |(\Delta\theta)_{i,j}|^2}. \quad (1)$$

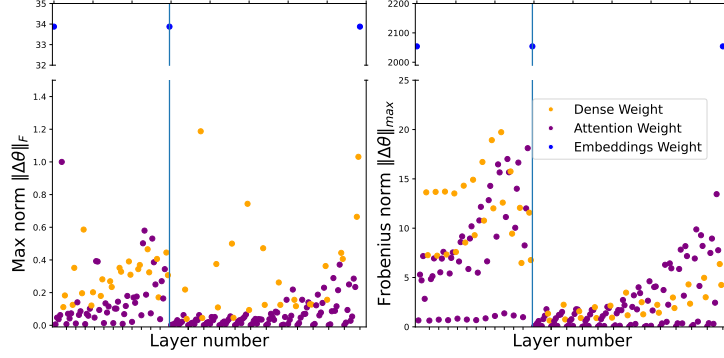


Fig. 1: Euclidean and Max norms differences between MonoT5 and T5 parameters. The vertical blue lines separate encoder and decoder parameters.

The Frobenius norm is inherently sensitive to matrix dimensionality: larger matrices tend to produce higher values as a consequence of their size. Parameter matrices of MonoT5 vary greatly in scale: for example, the embedding layer has dimensions 768×32128 , whereas attention weights are only 768×768 . As a result, the Frobenius norm of the embedding layer is expected to dominate due to its larger size. To obtain a measure of distance that is independent of matrix dimensionality, we instead apply the Max norm [5] to $\Delta\theta$, as follows:

$$\|\Delta\theta\|_{\max} = \max_{i,j} |(\Delta\theta)_{i,j}|. \quad (2)$$

We present the results of our analysis in Figure 1. We observe that the largest parameter shifts, measured with both the Frobenius and the Max norm, occur in the embedding weights of the encoder, the decoder, and the language modelling head. These three layers are tied in T5, i.e. they have the same weights, such that the model is learning a unique representation vector for each token on both the encoder and decoder embeddings and on the language modelling head. While the highest Frobenius norm of the embedding layers can be partly attributed to their large dimensionality, they also dominate under the Max norm, which shows that the most important updates are localised in these matrices. In contrast, both attention and dense layer parameters have Max norms below 1 and Frobenius norms below 20, indicating only marginal updates.

Observation 3.1

The training process in MonoT5 is dominated by changes in the embedding layer, with minimal adaptation occurring in the other parameters.

We now further analyse the embedding layers, to understand which tokens achieve the highest shift during the training of MonoT5. Given a token t , we apply the cosine similarity and the euclidean distance between $e_{MonoT5}^t \in \mathbb{R}^{768}$ and $e_{T5}^t \in \mathbb{R}^{768}$, which are the 768-dimensional embeddings of t of MonoT5 and

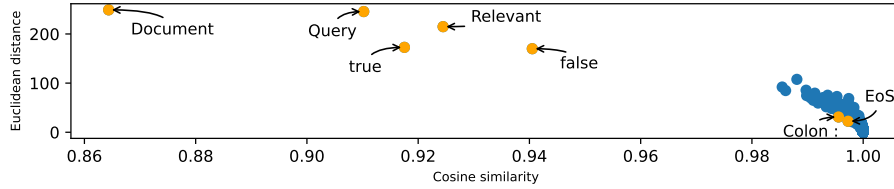


Fig. 2: Analysis of euclidean distance and cosine similarity between MonoT5 and T5 embedding vectors. Each point corresponds to a token in the T5/MonoT5 vocabulary. We highlight the tokens used in the prompt for training MonoT5.

T5, respectively. Using both cosine similarity and euclidean distance provides a complete characterisation of vector differences, as cosine captures directional alignment while euclidean reflects magnitude shifts.

The results of our analysis are presented in Figure 2. Broadly speaking, the embedding vectors exhibit high mutual similarity, with cosine similarity values close to 1. Among all tokens, the one with the lowest cosine similarity after fine-tuning is **Document**, with a value of 0.864. Figure 2 shows that the most affected embeddings are **Document**, **Relevant**, **Query**, **true**, and **false**. These correspond to the prompt tokens used to structure the input in MonoT5, as well as the target tokens (**true/false**) that the model generates to make a relevance prediction. Interestingly, within the top tokens showing the lowest cosine similarity, we also observe the colon symbol ‘:’, which, while less semantically important than the other prompt tokens, is part of the prompt template and thus one of the most influenced by fine-tuning.

Observation 3.2

MonoT5 primarily adapts the embeddings associated with prompt tokens, as they are trained to address task-specific instructions.

Observations 3.1 and 3.2 inspire us to investigate the minimal set of T5 model’s parameters needed to be trained for effective reranking performance.

4 Training Prompt Tokens is All You Need

In this section, we describe our proposed PEFT approach, which leads to the definition of a new T5-based re-ranker, which we call Light-MonoT5. We also combine Light-MonoT5 with QT5, which is a T5 trained model without queries, to reveal the learning mechanisms of MonoT5.

Formal Definition Let f_{θ} denote a pre-trained T5 model with parameters θ . Fine-tuning f_{θ} on a downstream task produces a new model $f_{\theta'}$ with parameters θ' . In our case, we focus on a ranking model f_{θ} that, given a set of texts $\mathcal{C} = \{d_i\}_{i=1}^{|\mathcal{C}|}$, and a query q , orders the documents by estimated relevance to q .

The ranking model f_{θ} is usually fine-tuned on triples $\langle q, d+, d- \rangle$, composed of a query q , a relevant document $d+ \in \mathcal{C}$ and a not-relevant document $d- \in \mathcal{C}$. Traditionally, fine-tuning f_{θ} requires updating all parameters θ . Instead, we consider a more granular perspective by partitioning the parameters into two disjoint subsets: the embedding parameters θ_{Emb} and all remaining parameters θ^c (e.g. attention and feed-forward layers), such that $\theta = \theta_{Emb} \cup \theta^c$.

Light-MonoT5 As stated by empirical Observations 3.1 and 3.2, the fine-tuning process of MonoT5 mostly affects the embeddings of the prompt tokens, with only minor changes occurring in the rest of the model’s parameters. This gives rise to the intuition that an effective MonoT5 may be achieved by training fewer parameters and thus reducing its training time. Specifically, during fine-tuning, we freeze all parameters θ^c and restrict updates to a small subset of embeddings within θ_{emb} . Concretely, this subset corresponds to the tokens used in the MonoT5 prompt: **Query**, **Document**, **Relevant**, ‘:’, **true**, **false**, [EOS], where EOS is the ‘*End of Sentence*’ token of T5. The motivation behind Light-MonoT5 is grounded in the nature of T5’s pre-training. T5 has been fine-tuned on a wide range of tasks, including question answering [25]. As a result, it already encodes strong question-answering abilities. Thus, fine-tuning the model for predicting relevance between a query and a document does not introduce an entirely new skill, but rather adapts and uncovers capabilities that are already latent in the model. For these reasons, we do not need to train all the parameters of T5, but by updating only the embeddings of prompt tokens, Light-MonoT5 effectively activates these latent skills and aligns them with the document ranking task.

Relevance vs. Quality The comparison between MonoT5 and Light-MonoT5 raises a fundamental question: *what is MonoT5 actually learning?*

If Light-MonoT5 achieves comparable performance by updating only a minimal subset of embeddings, then the full fine-tuning of MonoT5 may not be focused on modelling query–document relevance. Instead, we hypothesise that MonoT5 has learned to classify the quality of a passage and thus to discriminate between passages that are generally well-formed and informative versus those unlikely to be relevant to any query. To test this hypothesis, we employ Quality T5 (QT5) [7], which is a T5-based model trained to identify passages unlikely to be relevant to any query, as follows: (i) QT5 is applied to all passages p of a corpus $\mathcal{C} = \{p_1, \dots, p_n\}$ to obtain a quality score for each text; (ii) given a quality threshold t , all the passages whose quality score is below the threshold t are pruned; (iii) only the passages that have not been pruned in the previous step are indexed, i.e. all the passages in $\mathcal{C}' = \{p | p \in \mathcal{C} \wedge QT5(p) \geq t\}$; (iv) after indexing, we apply a first-stage ranker method and then our Light-MonoT5.

The application of QT5 allows us to disentangle relevance from document quality. If Light-MonoT5 combined with QT5 performs on par with MonoT5, it would support the hypothesis that MonoT5’s learning mechanism is largely driven by document quality estimation instead of query–document relevance.

5 Experimental Setup

Our experiments investigate the role of prompt tokens embedding in MonoT5 through the following research questions:

- RQ1: If we train only prompt tokens, do we achieve comparable results to MonoT5?
- (a) Does Light-MonoT5 achieve performance on par with MonoT5 on in-domain benchmarks?
 - (b) How well does Light-MonoT5 transfer to corpora of other domains?
- RQ2: What is the role of passage quality in MonoT5?

Model Details and Baselines To obtain Light-MonoT5, we fine-tune three versions of a T5 [25] backbone model⁵ over the MS MARCO’s official training triples. We fine-tune our model as the original MonoT5 paper [22] with a constant learning rate of 10^{-3} for 100K iterations with class-balanced batches of size 128. We use a maximum of 512 input tokens and two output tokens (one for the target token and another for the end-of-sequence token). Given a query-passage pair, both Light-MonoT5 and MonoT5 compute the relevance probabilities by applying the softmax only to `true` and `false` tokens. Furthermore, we rely on two publicly available versions of QT5 [7], i.e. QT5-small⁶ and -base⁷. Following the QT5 experimental setting [7], we train T5-large for 10k on MS MARCO triple without any query to define QT5-large.

As baselines, we employ a lexical ranker, namely BM25, and three different versions of MonoT5 rerankers [22], i.e. small⁸, base⁹ and large¹⁰. Furthermore, we compare our Light-MonoT5 with three PEFT methods: LoRA [14], which updates only the attention weights through low-rank matrices; LoRA + *Embedding layer*, which train both LoRA and all the embedding layer parameters; *Embedding layer*, which updates only all the embedding parameters. For all the LoRA-based models, we use a low-rank dimension $r = 8$ from the original LoRA study [14]. All these three baseline models are trained in the same setting as Light-MonoT5. To compare the efficiency of our approach, we report the number of parameters updated during the training process (*# Params.*) and training speed up compared to MonoT5.

Test Collections To evaluate our approach, we use three query sets based on the MS MARCO passage retrieval collection [21], which contains 8.8M short textual passages. In detail, we use TREC Deep Learning 2019 [10], 2020 [9] and Hard [19] to evaluate the ability of Light-MonoT5 of learning relevance while fine-tuning only a small set of parameters. For brevity, we henceforth refer to these datasets as DL 19, DL 20 and DL Hard, respectively. Specifically, we use the judged versions, available through the *ir-datasets* package [18]. For evaluating our domain adaptation solution and testing transferability to new domains while maintaining relevance assessments, we selected diverse target collections from the public BEIR benchmark [28], including domain-specific datasets such as NFCorpus [2],

⁵ t5-small, t5-base, t5-large

⁷ pyterrier-quality/qt5-base

⁹ castorini/monot5-base ¹⁰ castorini/monot5-large

⁶ pyterrier-quality/qt5-small

⁸ castorini/monot5-small-100k

TREC-COVID [30], SciFact [31], SciDocs [8], and FiQA [20]. These datasets cover a range of tasks: SciDocs and SciFact focus on scientific citation prediction and fact checking, TREC-Covid and NFCorpus address biomedical retrieval, and FiQA targets financial question answering.

Evaluation Metrics For all the datasets, we use nDCG@10 [11] to evaluate the performance of the proposed approach. In our settings, i.e. updating only a small subset of the model’s parameters, we are primarily concerned with whether results are of the same quality as updating all the parameters. For this reason, to establish statistical significance between our approach and MonoT5, we use both: (i) a Two One-Sided t-Test (TOST) [27] with $p < 0.05$ to test statistical equivalence between Light-MonoT5 and MonoT5; (ii) a paired t-test for statistical difference with $p < 0.05$ and Bonferroni correction for multiple tests [11].

6 Results

In this section, we answer our research questions outlined in Section 5 and provide further analysis of our proposed approach. In detail, we discuss the results of our new PEFT approach and our models Light-MonoT5. We report the results of our evaluation in Table 1 and 3. In each table, the model before $\gg$ is the first stage ranker, i.e. BM25, while the model after is a reranker model over the top 100 documents retrieved by BM25.

In the following, Section 6.1, addresses RQ1a, which asks whether we may achieve comparable in-domain performance to MonoT5 while updating only a restricted subset of token embeddings, while Section 6.2 addresses RQ2b, to study the impact of Light-MonoT5 on out-of-domain benchmarks. Finally, Section 6.3 investigates the role of passage quality in MonoT5’s learning mechanism.

6.1 RQ1(a): Does Light-MonoT5 achieve performance on par with MonoT5 on in-domain benchmarks?

We first evaluate Light-MonoT5-base on in-domain benchmarks, i.e. DL 19, DL 20 and DL Hard. To investigate the potential of minimal embedding updates, we design a sequence of controlled training experiments in which only selected token embeddings are updated while the rest of the model remains frozen. We begin by updating solely the embedding of one token, chosen from **Query**, **Document**, **Relevant**, or **EOS**. Then, we expand to pairs of tokens, i.e **Query** and **Document**, and the two output tokens, i.e. **true** and **false**. Finally, we incrementally add embeddings until we reach all the tokens in the MonoT5 prompt, namely **Query**, **Document**, and **Relevant**, **true**, **false**, **EOS** and ‘:’.

Table 1 reports the results in term nDCG@10 of all these configurations (BM25 $\gg$ Light-MonoT5) compared to BM25, T5 and MonoT5 reranker. The green check (✓) marks the embeddings that are updated, while the grey cross (✗) indicates frozen embeddings.

On analysing the table, we first note that applying T5 in a zero-shot manner performs poorly compared to BM25, highlighting that some degree of training

Table 1: Effectiveness in nDCG@10 and efficiency in # of parameters updated and training speed up of all proposed configurations of Light-MonoT5-base. * and † mean not statistically significant difference compared to MonoT5 with t-test and TOST-test, respectively.

Model								# Params.	Training Speed Up	DL 19	DL 20	DL Hard
BM25								/	/	.479	.494	.274
BM25 $\gg$ T5								0	0	.343	.231	.110
Query Document Relevant true false EoS Colon (:)												
BM25 $\gg$ Light-MonoT5	✓	×	×	×	×	×	×	768	1.50×	.610	.582	.335
	×	✓	×	×	×	×	×	768	1.50×	.555	.516	.298
	×	×	✓	×	×	×	×	768	1.50×	.547	.446	.243
	×	×	×	×	×	✓	×	768	1.50×	.363	.255	.135
	✓	✓	×	×	×	×	×	1536	1.48×	.669	.609	.351
	×	×	×	✓	✓	✓	×	1536	1.48×	.457	.381	.164
	✓	✓	✓	×	×	×	×	2304	1.49×	.681	.617	.344
	✓	✓	✓	×	×	✓	×	3072	1.48×	.664	.624	.337
	✓	✓	✓	✓	✓	✓	×	3840	1.47×	.686*	.639	.356
	✓	✓	✓	✓	✓	✓	×	4608	1.47×	.696* [†]	.631	.347
	✓	✓	✓	✓	✓	✓	✓	5376	1.46×	.694* [†]	.625	.350
	BM25 $\gg$ T5 + LoRA								885k	1.18×	.675	.630
BM25 $\gg$ T5 + LoRA + Embedding layer								25.6M	1.12×	.690	.594	.329
BM25 $\gg$ T5 + Embedding layer								24.7M	1.15×	.698* [†]	.640	.343
BM25 $\gg$ MonoT5								223M	1×	.701	.679	.400

is required. Updating only **Query** or **Document** outperforms BM25 on DL 19, DL 20, and DL Hard, showing that even minimal updates enable T5 to learn relevance, whereas updating only **Relevant** or **EoS** leads to markedly underperform, even worse than BM25. This observation aligns with recent findings [1,12], which suggest that pre-trained language models rely heavily on the initial instruction tokens, as, for example, T5 is fine-tuned with task-specific prompts defined at the beginning of the input [25]. When both **Query** and **Document** are updated jointly, performance further improves compared to updating either token alone. However, updating only **true** and **false**, despite training the same number of tokens, fails to outperform BM25, suggesting that relevance prediction cannot be effectively guided by output labels alone; instead, it requires training specific prompt tokens that define the task. Extending training to three tokens, **Query**, **Document**, and **Relevant**, yields even higher performance. Adding **EoS** slightly increases effectiveness on DL 20, but decreases performance on DL 19 and DL Hard. Notably, when training **Query**, **Document**, and **Relevant** together with **true** and **false**, Light-MonoT5 reaches nDCG@10 on DL 19 that is not significantly different from MonoT5-base, while requiring updates to only 4608 parameters, which is 0.002% of MonoT5 parameters, and 1.5× faster training. Finally, including **EoS** and/or the colon token further improves performance, reaching statistical equivalence with MonoT5-base under the TOST test on DL 19.¹¹ However, these configurations show weaker generalisation to DL 20 and DL Hard. For comparison, we also evaluate three PEFT baselines: LoRA, LoRA+ *Embedding*

¹¹ We further evaluate our approach by training three new special tokens instead of **Query**, **Document**, and **Relevant**, achieving performance on par with MonoT5.

Table 2: Effectiveness in nDCG@10 of all proposed configurations of Light-MonoT5 with TCT-ColBERT as first-stage ranker. Notation as per Table 1.

Model								# Params.	Training Speed Up	DL 19	DL 20	DL Hard	
TCT-ColBERT								/	/	.718	.683	.371	
TCT-ColBERT $\gg$ T5								0	0	.313	.224	.123	
	Query Document Relevant true false EoS Colon (:)												
TCT-ColBERT $\gg$ Light-MonoT5-base	✓	×	×	×	×	×	×	768	1.50×	.651	.590	.341	
	×	✓	×	×	×	×	×	768	1.50×	.563	.549	.306	
	×	×	×	✓	×	×	×	768	1.50×	.527	.451	.243	
	×	×	×	×	×	×	✓	768	1.50×	.338	.241	.121	
	✓	✓	×	×	×	×	×	1536	1.48×	.684*	.611	.347	
	×	×	×	✓	✓	✓	×	1536	1.48×	.438	.360	.174	
	✓	✓	✓	✓	×	×	×	2304	1.49×	.711*	.631	.365*	
	✓	✓	✓	✓	×	×	×	3072	1.48×	.691*	.637	.351	
	✓	✓	✓	✓	✓	✓	×	3840	1.47×	.691*	.637	.364	
	✓	✓	✓	✓	✓	✓	✓	4608	1.47×	.702*	.639	.375*	
	✓	✓	✓	✓	✓	✓	✓	5376	1.46×	.710*	.635	.360	
	TCT-ColBERT $\gg$ T5 + LoRA								885k	1.18×	.687*	.651	.375*
	TCT-ColBERT $\gg$ T5 + LoRA + Embedding layer								25.6M	1.12×	.670*	.608	.327
	TCT-ColBERT $\gg$ T5 + Embedding layer								24.7M	1.15×	.697*	.648	.370*
TCT-ColBERT $\gg$ MonoT5								223M	1×	.717	.700	.413	

layer, and *Embedding layer* alone. All the proposed baselines achieve competitive performance with Light-MonoT5, with the embedding-layer baseline showing particularly strong results on DL 20. Nevertheless, all require substantially more parameters and at least 1.3 $\times$ training time with respect to Light-MonoT5.

Furthermore, we extend our analysis to a stronger semantic first-stage retriever, namely TCT-ColBERT [17], to assess whether our findings generalise beyond a lexical first-stage retriever. The results of this analysis are reported in Table 2. Consistently with the trends observed with BM25, Light-MonoT5 achieves strong effectiveness across all three benchmarks (DL 19, 20 and Hard). In particular, when training the full set of prompt tokens, Light-MonoT5 achieves nDCG@10 scores that are statistically equivalent to those obtained by fully fine-tuned MonoT5. These results show that training only prompt-token embeddings is sufficient to capture the core relevance signals learned by MonoT5, even when the first-stage retriever already provides semantically rich passages.

To concretely answer RQ1(a), Light-MonoT5 achieve competitive performance compared to MonoT5 on in-domain benchmarks, notably on DL 19, where they are not statistically significantly different.

6.2 RQ1(b): How well does Light-MonoT5 transfer to corpora of other domains?

To understand if our approach can generalise well on different T5 variants and out-of-domain datasets, we train three versions of Light-MonoT5, i.e. small, base and large, and evaluate them on out-of-domain benchmarks. In detail, we train the embedding related to tokens **Query**, **Document**, and **Relevant** together with **true** and **false** and **EoS**, because it is the one that achieves the best performance on both DL 19 and DL 20. As shown in Table 3, Light-MonoT5 consistently

Table 3: Effectiveness in nDCG@10 of Light-MonoT5 on out-domain benchmarks. Notation for * and † as per Table 1.

Model	DL 19	DL 20	DL Hard	NFCorpus	SciDocs	SciFact	TREC-Covid	FiQA
BM25	.479	.494	.274	.328	.158	.684	.629	.253
BM25 $\gg$ Light-MonoT5-small	.629*	.540	.298	.319	.118	.628	.632	.277
BM25 $\gg$ MonoT5-small	.673	.640	.354	.334	.146	.672	.694	.344
BM25 $\gg$ Light-MonoT5-base	.696*†	.631	.347	.341*	.145	.707*	.723*	.354
BM25 $\gg$ MonoT5-base	.701	.679	.400	.347	.157	.720	.725	.386
BM25 $\gg$ Light-MonoT5-large	.685*	.665*	.350*	.359*	.162	.718	.729*	.391
BM25 $\gg$ MonoT5-large	.711	.685	.359	.367	.174	.737	.759	.421

performs on par, and in several cases without statistically significant differences, with fully fine-tuned MonoT5 across different model sizes. On the in-domain benchmarks such as DL 19, all three Light-MonoT5 variants achieve effectiveness that is statistically indistinguishable from their corresponding MonoT5 models. However, on out-of-domain benchmarks, only Light-MonoT5-base and Light-MonoT5-large maintain competitive performance, achieving statistical equivalence with MonoT5 on NFCorpus, SciFact, and TREC-Covid. These findings highlight that while our token-efficient approach is robust across domains, it can still be sensitive to datasets with greater semantic variability, such as the financial domain. To concretely answer RQ1(b), Light-MonoT5-base and large achieve statistical equivalence with MonoT5 on two of the five datasets, namely NFCorpus and TREC-Covid, but the small variant markedly underperforms, showing that our approach is sensitive to domain and vocabulary shift.

6.3 RQ2: What is the role of passage quality in MonoT5?

The performance of Light-MonoT5 shows that the T5 model can effectively and efficiently learn relevance judgments even when training is restricted to a small, carefully selected subset of its parameters on the MS MARCO dataset. This observation suggests that full fine-tuning of all parameters may not be needed for capturing relevance signals. Indeed, the generalisation ability of T5 can be partly attributed to its prior fine-tuning on different supervised tasks such as question answering (e.g., SQuAD [26]) and semantic similarity (e.g., STS-B [6]). These tasks implicitly encode aspects of semantic relation and context alignment, which overlap with the notion of query–document relevance. Consequently, T5 already incorporates the ability to capture relevance, even without full fine-tuning on MS MARCO. Based on this insight, our RQ2 investigates what MonoT5 is learning during training on MS MARCO. Unlike Light-MonoT5, MonoT5 requires updating the entire set of 223M parameters, raising the possibility that the model’s performance may not derive from learning fine-grained query–document relevance, but rather from learning to classify passage quality. To test our hypothesis, we apply QT5 as a document quality filter, pruning low-quality passages from MS MARCO before indexing. By subsequently applying Light-MonoT5 on this pruned corpus, we expect to observe performance that is comparable to MonoT5,

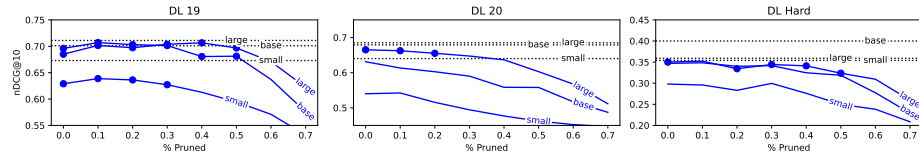


Fig. 3: nDCG@10 of Light-MonoT5 by the percentage of a corpus pruned by using QT5. Effectiveness measurements that are not statistically significantly different from MonoT5 are marked with •.

thereby supporting the claim that full fine-tuning mainly helps the model discriminate document quality. Following Chen et al. [7], we prune up to 70% of MS MARCO passages, since when pruning to over 70%, the size of the lexicon begins to have a substantial impact on the total index size for MS MARCO. Our results are shown in Figure 3. We note that both Light-MonoT5-base and -large achieve performance on DL 2019 that is not significantly different from their fully fine-tuned MonoT5 counterparts, even when up to 50% of the MS MARCO passages are pruned. A similar trend holds on DL Hard, particularly for the base variant. However, Light-MonoT5-small achieve performance equivalence with MonoT5 only on DL 20 with pruning levels up to 30%, suggesting that smaller variants may effectively learn relevance signals better than bigger ones. These observations show that if we prune low-quality passages, we do not need to update all of MonoT5’s parameters to be able to find the highly relevant passages. These findings reinforce our hypothesis: when fine-tuning MonoT5, the model has learned how to classify document quality, rather than query-token relevance, answering RQ2.

7 Conclusions

In this work, we show that fine-tuning MonoT5 mainly updates prompt token embeddings rather than the full model. Building on this, we introduce Light-MonoT5, a parameter-efficient variant that updates only 0.002% of parameters while preserving effectiveness across benchmarks. These findings provide new insight into MonoT5’s training mechanisms, showing that MonoT5 learns to classify document quality rather than query-passage relevance. Future work may extend this approach to multilingual settings and different large language models.

Acknowledgments. Braga acknowledges the CINECA award under the IS-CRA initiative, for the availability of high-performance computing resources and support.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Barbero, F., Arroyo, A., Gu, X., Perivolaropoulos, C., Veličković, P., Pascanu, R., Bronstein, M.M.: Why do LLMs attend to the first token? In: Second Conference on Language Modeling (2025)
2. Boteva, V., Gholipour, D., Sokolov, A., Riezler, S.: A full-text learning to rank dataset for medical information retrieval. In: European Conference on Information Retrieval. pp. 716–722. Springer (2016)
3. Braga, M.: Personalized large language models through parameter efficient fine-tuning techniques. In: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 3076–3076 (2024)
4. Braga, M., Kasela, P., Raganato, A., Pasi, G.: Investigating task arithmetic for zero-shot information retrieval. In: Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval. p. 2738–2743. SIGIR ’25, Association for Computing Machinery, New York, NY, USA (2025)
5. Cai, T., Zhou, W.X.: A max-norm constrained minimization approach to 1-bit matrix completion. *Journal of Machine Learning Research* **14**, 3619–3647 (2013)
6. Cer, D., Diab, M., Agirre, E., Lopez-Gazpio, I., Specia, L.: SemEval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. In: Bethard, S., Carpuat, M., Apidianaki, M., Mohammad, S.M., Cer, D., Jurgens, D. (eds.) Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017). pp. 1–14. Association for Computational Linguistics, Vancouver, Canada (Aug 2017), <https://aclanthology.org/S17-2001/>
7. Chang, X., Mishra, D., Macdonald, C., MacAvaney, S.: Neural passage quality estimation for static pruning. In: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval. p. 174–185. SIGIR ’24, Association for Computing Machinery, New York, NY, USA (2024), <https://doi.org/10.1145/3626772.3657765>
8. Cohan, A., Feldman, S., Beltagy, I., Downey, D., Weld, D.: SPECTER: Document-level representation learning using citation-informed transformers. In: Jurafsky, D., Chai, J., Schluter, N., Tetreault, J. (eds.) Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. pp. 2270–2282. Association for Computational Linguistics, Online (Jul 2020), <https://aclanthology.org/2020.acl-main.207/>
9. Craswell, N., Mitra, B., Yilmaz, E., Campos, D.: Overview of the trec 2020 deep learning track (2021), <https://arxiv.org/abs/2102.07662>
10. Craswell, N., Mitra, B., Yilmaz, E., Campos, D., Voorhees, E.M.: Overview of the trec 2019 deep learning track (2020), <https://arxiv.org/abs/2003.07820>
11. Dunn, O.J.: Multiple comparisons among means. *Journal of the American Statistical Association* **56**, 52–64 (1961)
12. Gu, X., Pang, T., Du, C., Liu, Q., Zhang, F., Du, C., Wang, Y., Lin, M.: When attention sink emerges in language models: An empirical view. In: The Thirteenth International Conference on Learning Representations (2025)
13. Herzog, R., Köhne, F., Kreis, L., Schiela, A.: Frobenius-type norms and inner products of matrices and linear maps with applications to neural network training. arXiv preprint arXiv:2311.15419 (2023)
14. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al.: Lora: Low-rank adaptation of large language models. *ICLR* **1**(2), 3 (2022)

15. Humeau, S., Shuster, K., Lachaux, M.A., Weston, J.: Poly-encoders: Transformer architectures and pre-training strategies for fast and accurate multi-sentence scoring. arXiv preprint arXiv:1905.01969 (2019)
16. Ilharco, G., Ribeiro, M.T., Wortsman, M., Schmidt, L., Hajishirzi, H., Farhadi, A.: Editing models with task arithmetic. In: The Eleventh International Conference on Learning Representations (2022)
17. Lin, S.C., Yang, J.H., Lin, J.: Distilling dense representations for ranking using tightly-coupled teachers. arXiv preprint arXiv:2010.11386 (2020)
18. MacAvaney, S., Yates, A., Feldman, S., Downey, D., Cohan, A., Goharian, N.: Simplified data wrangling with `ir_datasets`. In: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 2429–2436. SIGIR '21, Association for Computing Machinery, New York, NY, USA (2021), <https://doi.org/10.1145/3404835.3463254>
19. Mackie, I., Dalton, J., Yates, A.: How deep is your learning: the dl-hard annotated deep learning dataset. In: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (2021)
20. Maia, M., Handschuh, S., Freitas, A., Davis, B., McDermott, R., Zarrouk, M., Balahur, A.: Wwv'18 open challenge: financial opinion mining and question answering. In: Companion proceedings of the the web conference 2018. pp. 1941–1942 (2018)
21. Nguyen, T., Rosenberg, M., Song, X., Gao, J., Tiwary, S., Majumder, R., Deng, L.: MS MARCO: A human generated machine reading comprehension dataset. In: Besold, T.R., Bordes, A., d'Avila Garcez, A.S., Wayne, G. (eds.) Proceedings of the Workshop on Cognitive Computation: Integrating neural and symbolic approaches 2016 co-located with the 30th Annual Conference on Neural Information Processing Systems (NIPS 2016), Barcelona, Spain, December 9, 2016. CEUR Workshop Proceedings, vol. 1773. CEUR-WS.org (2016)
22. Nogueira, R., Jiang, Z., Pradeep, R., Lin, J.: Document ranking with a pretrained sequence-to-sequence model. In: Cohn, T., He, Y., Liu, Y. (eds.) Findings of the Association for Computational Linguistics: EMNLP 2020. pp. 708–718. Association for Computational Linguistics, Online (Nov 2020), <https://aclanthology.org/2020.findings-emnlp.63/>
23. Nogueira, R., Yang, W., Cho, K., Lin, J.: Multi-stage document ranking with bert. arXiv preprint arXiv:1910.14424 (2019)
24. Parry, A., Fröbe, M., MacAvaney, S., Potthast, M., Hagen, M.: Analyzing adversarial attacks on sequence-to-sequence relevance models. In: European Conference on Information Retrieval. pp. 286–302. Springer (2024)
25. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J.: Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* **21**(140), 1–67 (2020)
26. Rajpurkar, P., Zhang, J., Lopyrev, K., Liang, P.: SQuAD: 100,000+ questions for machine comprehension of text. In: Su, J., Duh, K., Carreras, X. (eds.) Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing. pp. 2383–2392. Association for Computational Linguistics, Austin, Texas (Nov 2016), <https://aclanthology.org/D16-1264/>
27. Schuirmann, D.J.: A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of pharmacokinetics and biopharmaceutics* **15**(6), 657–680 (1987)
28. Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., Gurevych, I.: BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In: Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2) (2021)

29. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. *Advances in neural information processing systems* **30** (2017)
30. Voorhees, E., Alam, T., Bedrick, S., Demner-Fushman, D., Hersh, W.R., Lo, K., Roberts, K., Soboroff, I., Wang, L.L.: Trec-covid: constructing a pandemic information retrieval test collection. In: *ACM SIGIR Forum*. vol. 54, pp. 1–12. ACM New York, NY, USA (2021)
31. Wadden, D., Lin, S., Lo, K., Wang, L.L., van Zuylen, M., Cohan, A., Hajishirzi, H.: Fact or fiction: Verifying scientific claims. In: Webber, B., Cohn, T., He, Y., Liu, Y. (eds.) *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. pp. 7534–7550. Association for Computational Linguistics, Online (Nov 2020), <https://aclanthology.org/2020.emnlp-main.609/>